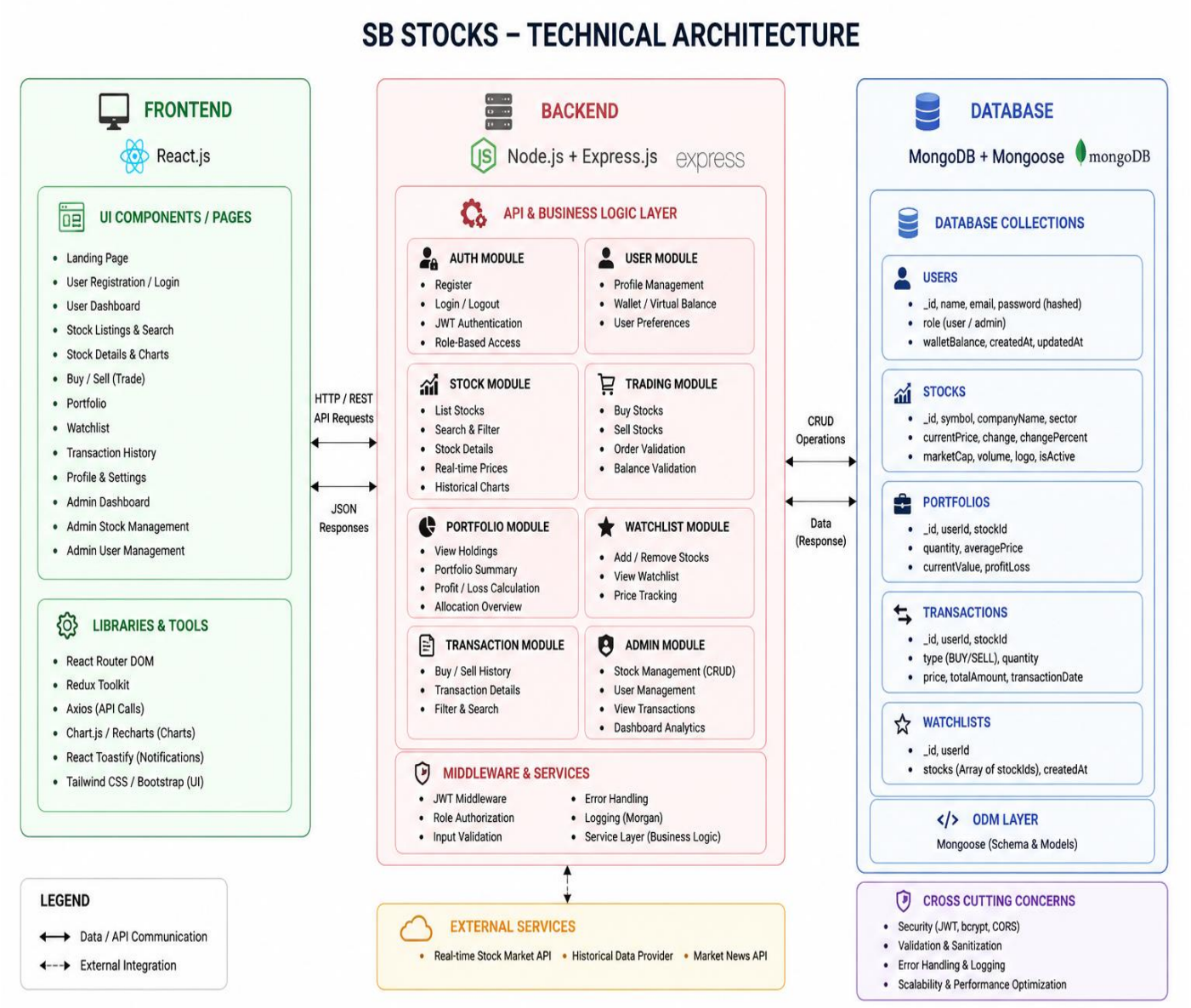
TECHNICAL ARCHITECTURE



1. FRONTEND (The Client Layer)

This section represents what the user interacts with directly in their browser.

Core Technology: Built using React.js.

UI Components / Pages: Lists all the distinct screens and features the user will see, such as:

- Landing Page
- User/Admin Dashboards
- Stock Listings
- Buy/Sell trading interfaces
- Portfolio management screens

Libraries & Tools: Details the supporting frontend ecosystem, including:

- Redux Toolkit for state management
- Axios for making API calls
- Chart.js / Recharts for visualizing stock data
- Tailwind CSS / Bootstrap for UI styling

2. BACKEND (The Server Layer)

This section acts as the brain of the application, processing requests and enforcing business rules.

Core Technology: Powered by Node.js and Express.js.

API & Business Logic Layer: This is highly modularized into eight distinct functional blocks:

- **Auth & User Modules:** Handles secure registration, login (via JWT), and profile/wallet management.
- **Stock & Trading Modules:** Manages stock searching, real-time pricing, executing buy/sell orders, and validating virtual balances.
- **Portfolio & Watchlist Modules:** Calculates profit/loss, tracks user holdings, and manages saved stocks.
- **Transaction & Admin Modules:** Logs trade history and provides system administrators with CRUD (Create, Read, Update, Delete) controls over users and stocks.

Middleware & Services: Handles background processes like JWT verification, role-based authorization, error handling, and request logging.

3. DATABASE (The Data Storage Layer)

This section shows where and how the application's permanent data is stored.

Core Technology: Utilizes MongoDB (a NoSQL database) and Mongoose (an Object Data Modeling library).

Database Collections: Outlines the exact schemas for the core entities:

- **Users:** Stores credentials, roles, and wallet balances.
- **Stocks:** Stores company details, symbols, and pricing.
- **Portfolios:** Links users to their specific stock holdings and quantities.
- **Transactions:** A ledger recording the specifics (price, type, date) of every trade.
- **Watchlists:** Arrays of specific stocks saved by users.

4. Communication Flow & Global Architecture

The diagram uses arrows to illustrate how these three pillars communicate:

- **Frontend to Backend:** They communicate via standard HTTP/REST API Requests and receive JSON Responses.
- **Backend to Database:** They communicate via standard CRUD Operations and data responses.
- **External Services:** Shows that the backend relies on external, third-party APIs to fetch real-time market data, historical charts, and market news.
- **Cross Cutting Concerns:** Highlights the global, non-functional priorities of the entire system, such as overarching security (JWT, bcrypt), strict data validation, and scalability optimization.